

5735.26 Deep Learning V26

April 26, 2026

Contents

1	Lecture 1: Foundations of Deep Learning	2
1.1	Motivation and Application Areas	2
1.2	Supervised Learning Setup	2
1.3	Model Family and Linear Regression	2
1.4	Training Objective and Loss Function	3
1.5	Gradient-Based Optimization	3
1.6	From Shallow Neural Networks to Piecewise Linear Models	3
1.7	Diagram: Accumulation of ReLU-Based Linear Pieces	4
1.8	Diagram: Input-Hidden-Output Architecture	4
2	Lecture 2: From Shallow to Deep Neural Networks	5
2.1	Supervised Learning Recap	5
2.2	Shallow Neural Network (One Hidden Layer)	5
2.3	Three Activation-Related Graphs	6
2.4	Perceptron, XOR, and Why Hidden Units Matter	6
2.5	Matrix Notation for the XOR Example	7
2.6	Vector Form for Multiple Hidden Units	7
2.7	Deep Neural Networks	7
2.8	Linear Region Capacity: Shallow vs Deep	8

1 Lecture 1: Foundations of Deep Learning

1.1 Motivation and Application Areas

Deep learning is a family of machine learning methods used in many master-level applications:

- Computer Vision
- Natural Language Processing (NLP)
- Speech and audio processing
- Games and intelligent agents

The key idea is to learn a function from data rather than hand-crafting rules.

1.2 Supervised Learning Setup

In supervised learning, we observe a dataset

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_I, y_I)\},$$

where $\mathbf{x}_i$ is an input and y_i is the target label/value.

We aim to learn a function

$$f : \mathbf{x} \mapsto y.$$

Examples:

- Scalar input example: $x = \text{height}$, $y = \text{age category}$.
- Medical imaging example:

$$\mathbf{x} = \text{MRI image vector} = [200, 310, \dots, 0], \quad y \in \{\text{Alzheimer: yes}, \text{Alzheimer: no}\}.$$

For each training example, the model prediction is

$$\hat{y}_i = f(\mathbf{x}_i; \phi),$$

where ϕ are model parameters.

1.3 Model Family and Linear Regression

We do not search over all possible functions; we choose a hypothesis/model family. A simple family is linear regression:

$$f(x; \phi) = \phi_0 + \phi_1 x.$$

For classification, $f(\mathbf{x}_0; \phi)$ can be interpreted (after a suitable output transform) as the probability of a class, e.g., Alzheimer diagnosis.

1.4 Training Objective and Loss Function

To select the best parameters, we minimize a loss on the dataset. For squared error:

$$L(\mathcal{D}, \phi) = \sum_{i=1}^I (f(\mathbf{x}_i; \phi) - y_i)^2.$$

This is the *sum of squares* criterion.

1.5 Gradient-Based Optimization

Parameters are updated with gradient descent:

$$\phi \leftarrow \phi - \alpha \nabla_{\phi} L(\phi),$$

where $\alpha > 0$ is the learning rate.

Directional derivative intuition: for a unit direction vector $\mathbf{v}$, the rate of change is

$$\mathbf{v}^{\top} \nabla_{\phi} L.$$

If the angle θ between $\mathbf{v}$ and ∇L satisfies $\cos \theta = 1$, change is maximally increasing; if $\cos \theta = -1$, change is maximally decreasing.

1.6 From Shallow Neural Networks to Piecewise Linear Models

A shallow neural network with one hidden layer and ReLU activations:

$$h_j = a(z_j), \quad z_j = \theta_{j0} + \theta_{j1}x, \quad a(z) = \max(0, z).$$

With two hidden units:

$$f(x; \phi) = \phi_0 + \phi_1 h_1 + \phi_2 h_2 = \phi_0 + \phi_1 a(\theta_{10} + \theta_{11}x) + \phi_2 a(\theta_{20} + \theta_{21}x).$$

One possible parameter set is

$$\phi = \{\phi_0, \phi_1, \phi_2, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}\}.$$

Each hidden unit introduces a kink at $\theta_{j0} + \theta_{j1}x = 0$. With D hidden units in 1D input, the model can form up to $D + 1$ linear regions.

1.7 Diagram: Accumulation of ReLU-Based Linear Pieces

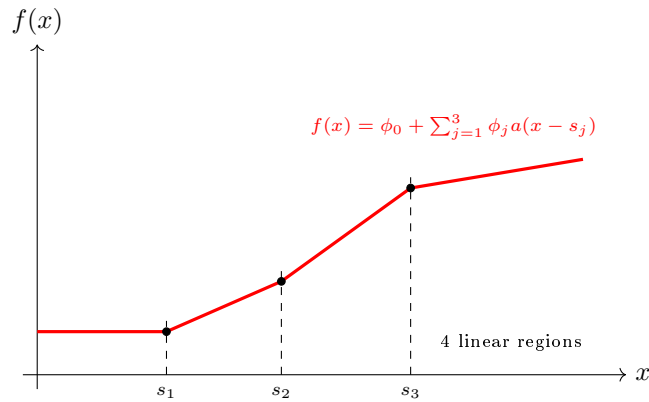


Figure 1: Combination of three shifted linear components producing three joints (kinks) in the final piecewise-linear function.

1.8 Diagram: Input-Hidden-Output Architecture

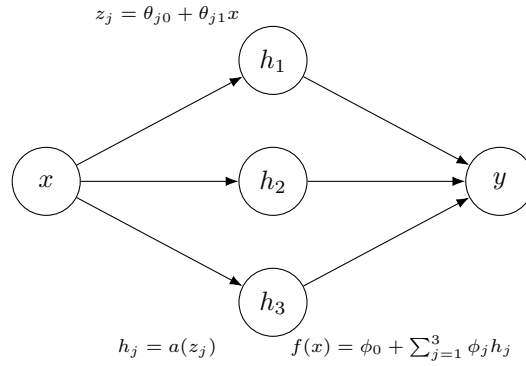


Figure 2: Shallow network with one input, three hidden ReLU units, and one output.

2 Lecture 2: From Shallow to Deep Neural Networks

2.1 Supervised Learning Recap

We mostly work with supervised learning:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I,$$

and we train a model that predicts

$$\hat{y}_i = f(\mathbf{x}_i; \phi).$$

The training objective is a loss over the dataset:

$$L(\mathcal{D}, \phi) = L(\phi).$$

2.2 Shallow Neural Network (One Hidden Layer)

For 1D input, a shallow ReLU network can be written as

$$f(x; \phi) = \phi_0 + \sum_{d=1}^D \phi_d h_d, \quad h_d = a(\theta_{d0} + \theta_{d1}x),$$

with ReLU activation

$$a(z) = \max(z, 0).$$

2.3 Three Activation-Related Graphs

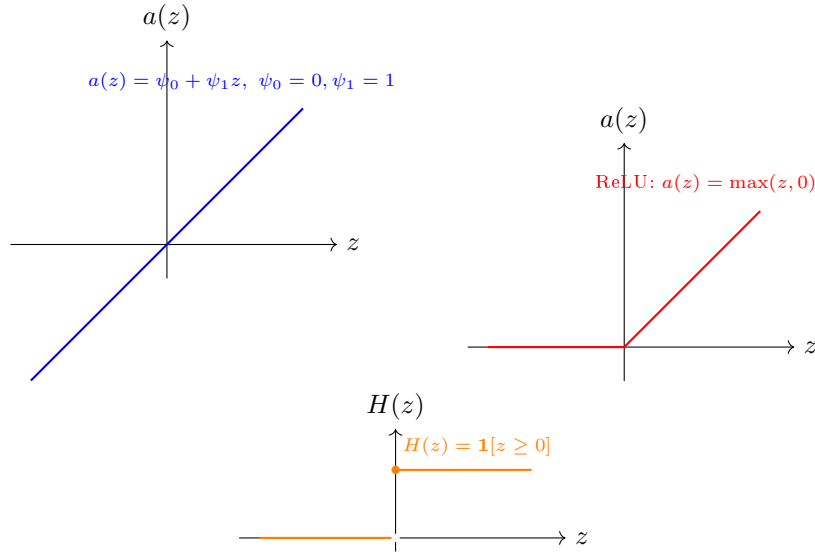


Figure 3: Left: linear activation. Middle: ReLU. Right: Heaviside step used in perceptron-style classifiers.

2.4 Perceptron, XOR, and Why Hidden Units Matter

A single linear perceptron creates one decision boundary, so XOR cannot be solved with one linear unit. With hidden units and gradient-based learning, we can represent XOR.

For the specific construction:

$$h_1 = a(0 + x_1 + x_2), \quad h_2 = a(-1 + x_1 + x_2), \quad y = 0 + h_1 - 2h_2.$$

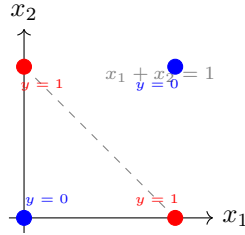


Figure 4: XOR pattern in input space: not linearly separable by one perceptron.

2.5 Matrix Notation for the XOR Example

Let

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

Then hidden preactivation and hidden state are

$$\mathbf{z} = W_1 \mathbf{x} + \mathbf{b}_1 = \begin{bmatrix} x_1 + x_2 \\ x_1 + x_2 - 1 \end{bmatrix}, \quad \mathbf{h} = a(\mathbf{z}) = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}.$$

Output:

$$y = W_2 \mathbf{h} + b_2, \quad W_2 = \begin{bmatrix} 1 & -2 \end{bmatrix}, \quad b_2 = 0.$$

2.6 Vector Form for Multiple Hidden Units

For three hidden units, write

$$\mathbf{h} = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = a \left(\begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \end{bmatrix} + \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{31} \end{bmatrix} x \right).$$

In expanded form:

$$\mathbf{h} = \begin{bmatrix} a(\theta_{10} + \theta_{11}x) \\ a(\theta_{20} + \theta_{21}x) \\ a(\theta_{30} + \theta_{31}x) \end{bmatrix}.$$

Then

$$y = \phi_0 + \begin{bmatrix} \phi_1 & \phi_2 & \phi_3 \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}.$$

2.7 Deep Neural Networks

A network is called *deep* when it has more than one hidden layer.

Layer-wise notation:

$$\mathbf{h}^{(1)} = a(\boldsymbol{\beta}_0 + \Omega_0 \mathbf{x}), \quad \mathbf{h}^{(2)} = a(\boldsymbol{\beta}_1 + \Omega_1 \mathbf{h}^{(1)}), \quad \dots, \quad \mathbf{h}^{(K)} = a(\boldsymbol{\beta}_{K-1} + \Omega_{K-1} \mathbf{h}^{(K-1)}),$$

$$\mathbf{y} = \boldsymbol{\beta}_K + \Omega_K \mathbf{h}^{(K)}, \quad \Phi = \{(\boldsymbol{\beta}_k, \Omega_k)\}_{k=0}^K.$$

Hyperparameters:

- K : number of hidden layers
- D_k : number of hidden units in layer k

Training still uses gradient descent to update weights and biases.

2.8 Linear Region Capacity: Shallow vs Deep

For 1D input, a shallow ReLU network with D hidden units has at most $D + 1$ linear regions.

For deep ReLU networks (input dimension d_0), a standard lower-bound style expression is

$$R_{\text{deep}} \gtrsim \left(\prod_{k=1}^{K-1} \left\lfloor \frac{D_k}{d_0} \right\rfloor^{d_0} \right) \sum_{j=0}^{d_0} \binom{D_K}{j}.$$

This captures why depth can create many more linear regions than a single hidden layer.